

VULNIGO

LITE EDITION

A gamified way to learn and test repository security.

Product Requirements, Workflow, Tech Stack, System Blueprint & Features

TEAM OF 6

Ishaan | Mariam | Askeri | APN | Ahmad | Sharjeel

3 AI agents

Red | Blue | Manager

2 main rounds

Attack | Defense

Human-in-the-loop

Optional participation

4-day target

Focused MVP scope

Contents

01 Product Requirements Document (PRD)
Vision, problem, users, goals, requirements, metrics

02 VULNIGO Lite Features
What ships in the MVP and what is deferred

03 Project Workflow
User journey, participation modes, four-day build plan

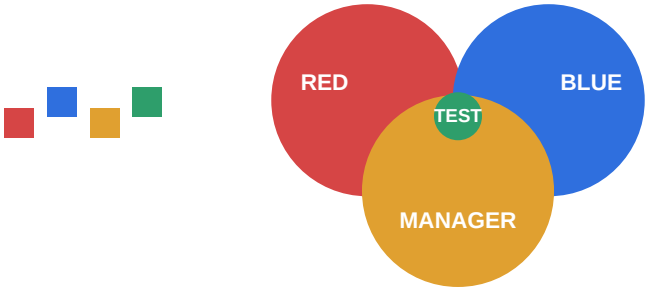
04 Team and Responsibilities
Six members, workstream owners and day-by-day assignments

05 Workflow Flowchart
End-to-end battle flow with every decision gate

06 Tech Stack
Proposed technologies per layer

07 System Blueprint
Architecture, agents, state machine, API, data, security, testing

Guiding principle. Build one complete, evidence-backed battle before adding more agents, vulnerability categories, matches, achievements or elaborate game mechanics. One successful attack. One meaningful defense. One independently verified fix.



Attack and defense, coordinated by the Manager, verified by independent tests.

01 Product Requirements Document

Scope: VULNIGO Lite MVP, four-day build target

Product overview

VULNIGO Lite is a human-in-the-loop, gamified repository-security experience. A user provides a GitHub repository; a **Red Agent** investigates a selected weakness and safely validates it, a **Blue Agent** explains and remediates it, and a **Manager Agent** coordinates the battle and compiles evidence. Users can spectate, join either team, or request AI backup. Final verdicts come from independent test evidence, never from an agent's own claim.

Problem statement

Repository security is hard to learn and easy to over-claim. Scanners produce unverified alerts, security training is passive, and AI-generated patches are often accepted without proof they work. Learners and small teams lack an engaging, safe way to see an attack, a defense and a verified retest in one loop.

Target users

User	Need	How VULNIGO helps
Student / learner	Understand how vulnerabilities are found and fixed	Join Blue or Red and learn alongside the agents; request backup when stuck
Developer	Check whether a repo has a real, exploitable issue and whether a fix works	Evidence-backed finding, reviewable patch diff, before/after retest
Team lead / reviewer	Trust what the tool claims	Strict status vocabulary, logged commands, approval gates, honest reporting
Demo audience / judges	See a complete story quickly	Pixel-art command centre with visible agent activity and a final report

Goals

- Accept a GitHub repository and inspect its source in a controlled environment.
- Demonstrate one known vulnerability in a deliberately vulnerable test repository.
- Generate a remediation proposal and require human approval before applying it.
- Retest the original issue and report what the evidence actually shows.
- Allow users to spectate or participate with either team, including optional AI backup.
- Make agent activity and battle state visible through the existing pixel-art command centre.

Non-goals (four-day MVP)

- Eight independent agents, six matches, multi-model debates or tournament systems.
- Support for every language, framework, repository size or vulnerability category.
- Unrestricted autonomous exploitation, public-target scanning or unsupervised patch deployment.
- Claims that a repository is completely secure from a single successful test.
- Learning curriculum, achievement economy, user accounts or production multi-tenant service.

Functional requirements

ID	Requirement	Priority
FR-1	Submit a GitHub repo URL (or pick the prepared demo repo); validate it and reject local-file, private-network and unsupported schemes.	Must
FR-2	Clone a read-only snapshot, record commit hash and basic project metadata.	Must
FR-3	Create a disposable, restricted sandbox for all analysis and code execution.	Must
FR-4	Red Agent proposes a candidate finding with file, line range, data flow and a bounded safe validation test.	Must
FR-5	Distinguish confirmed findings from unconfirmed hypotheses using recorded test evidence.	Must
FR-6	Blue Agent explains root cause and produces a reviewable patch diff.	Must
FR-7	Patch is applied to the sandbox working copy only after explicit human approval (or configured policy).	Must
FR-8	Retest the original safe test plus regression checks; report pass / fail / inconclusive.	Must
FR-9	Spectator, Join Red and Join Blue participation modes; spectator is a first-class path.	Must
FR-10	Need Backup: send the active finding, code context and conversation state to the Blue Agent for help.	Must
FR-11	Manager compiles timeline, findings, patch outcome, unresolved issues and score summary from stored facts.	Must
FR-12	Stream agent events and state changes to the UI; clearly show running / waiting / failed / completed.	Must
FR-13	Human messages and suggestions are injected into the active team's context.	Should
FR-14	Clearly labelled deterministic or pre-recorded fallback if a live model call fails.	Should

Non-functional requirements

Area	Requirement
Safety	Untrusted code never runs on the host; non-root, no secrets, no host sockets, no outbound network by default, CPU/memory/process/time limits.
Integrity	Model output is untrusted input; commands map to an allowlisted tool interface; no fake findings, approvals or fixes on failure.
Honesty	An inconclusive retest is reported as inconclusive, never as fixed. Reports match stored evidence.
Performance	Bounded model calls and concise context; backend code (not the Manager model) handles routine transitions.
Reliability	Same end-to-end demo is repeatable; timeouts, clone failures and API errors handled gracefully.
Auditability	Log commands, approvals, patches and test outcomes; clean up sandbox after the battle.

Success metrics

- A prepared repository completes a full attack-defense-retest cycle end to end, repeatedly.
- 100% of reported findings and scores trace to a recorded test result or source location.
- Patch application never occurs without an approval record.
- Zero known sandbox escapes or approval bypasses at demo freeze.
- A user can complete the flow in Spectator mode without being forced to respond at any step.

Acceptance definition. VULNIGO Lite is demo-ready when a prepared repository passes a complete attack-defense-retest cycle; the user can spectate or participate; patching requires approval; test evidence is retained; and the final report accurately describes what was and was not verified.

02 VULNIGO Lite Features

The minimum complete product, and what is intentionally deferred

Feature	Description
Repository intake	GitHub URL or prepared demo repo; validation, commit hash recorded, project metadata detected.
Sandboxed analysis	Disposable isolated environment with strict resource, filesystem and network limits.
Red Agent (attacker)	Finds a candidate weakness in one vulnerability class and runs a safe, bounded validation test locally.
Blue Agent (defender)	Explains root cause, proposes the smallest relevant patch and responds to human suggestions.
Manager Agent	Tracks battle state, routes context, links claims to evidence and writes the final report.
Two rounds	Round 1 Attack; Round 2 Defense, including retest of the original exploit check.
Participation modes	Spectator, Join Red, Join Blue. Participation is always optional.
Need Backup (Blue)	Sends finding, code context and conversation state to the Blue Agent for guidance; never bypasses approval.
Patch approval gate	User reviews the diff and approves or rejects; no silent changes.
Independent retest	Deterministic checks plus the original safe test; final status comes from test evidence.
Pixel-art command centre	Existing UI shows agent activity, battle state, chat, approvals, results and scoreboard.
Evidence-based report	Timeline, findings, patch outcome, unresolved risks and score summary from recorded facts.

UI example (reference only)

THIS IS AN EXAMPLE OF A UI. IT IS NOT THE UI THAT IS NEEDED IN VULNIGO LITE.



Example of a top-down pixel-art office style. Layout, characters and rooms are illustrative only.

Participation modes

Mode	What happens	Human involvement
Spectator	Red and Blue agents run the workflow autonomously within approval gates.	Watches progress; approves important actions such as patch application.
Join Red Team	User collaborates with the Red Agent on investigation and attack strategy.	Submits hypotheses, inspects evidence, suggests safe test ideas.
Join Blue Team	User collaborates with the Blue Agent on analysis and remediation.	Investigates, suggests a fix, reviews the patch, requests backup.

Deferred beyond Lite

In Lite	Deferred until later
One Red Agent and one Blue Agent	Multiple specialized agents per team
One Manager Agent role	Complex multi-agent negotiation
Two rounds: attack and defense	Six or more independent matches
One supported vulnerability class	Broad language / framework coverage
Spectator and optional team participation	Competitive multiplayer and tournaments
Blue Team Need Backup	Advanced tutoring paths, adaptive curriculum
Patch approval and independent retest	Automatic deployment to real environments
Basic evidence-based scoreboard / report	XP, streaks, achievements, public rankings

03 Project Workflow

How a user moves through a battle, and how the team builds it in four days

User journey

#	Step	What happens
1	Start	User submits a GitHub repository URL or selects the prepared demo repository.
2	Prepare	Backend validates the URL, retrieves the repo, identifies project structure and creates a restricted sandbox.
3	Choose participation	Spectator, Join Red Team or Join Blue Team.
4	Attack round	Red Agent investigates scope, proposes an attack path and attempts safe validation.
5	Finding review	System separates confirmed findings from unconfirmed hypotheses and records evidence.
6	Defense round	Blue Agent analyses the confirmed finding, explains the cause and proposes a patch; Need Backup available.
7	Approval	User reviews and approves the patch before it is applied in the sandbox.
8	Retest	Deterministic checks and the original safe test run again; Red may retry; final status comes from test evidence.
9	Results	Manager compiles timeline, findings, patch outcome, unresolved issues and score summary.

Optional participation rule. The user is never required to respond at every step. The system pauses only at explicitly configured approval points; otherwise agents continue within the safe, pre-authorized scope.

Stage responsibilities

Stage	Responsible	Output / gate
Repository intake	Backend	Validated repo reference, source snapshot, metadata
Environment setup	Sandbox controller	Restricted environment, limits, permitted test commands
Round 1 - Attack	Red Agent + optional human	Candidate vulnerability, hypothesis, safe validation evidence
Finding review	Manager + deterministic checks	Confirmed / unconfirmed / unable-to-validate
Round 2 - Defense	Blue Agent + optional human	Remediation proposal and patch diff awaiting approval
Patch approval	Human / approval policy	Approved or rejected; no silent changes
Retest	Independent tests + Red retest	Pass / fail / inconclusive and regression outcome
Final report	Manager + backend	Evidence-backed summary, unresolved risks, scores

Four-day build plan

Each day has a tangible exit condition. If one is missed, reduce optional scope rather than carrying multiple unfinished components forward.

Day	Objective	Key tasks	Exit criteria
Day 1 Foundation	Submit, inspect and display a repo without running untrusted code on the host	Backend skeleton and health endpoint; connect UI to status updates; GitHub URL validation and retrieval; record commit hash; create restricted sandbox; battle state storage and event reporting	Prepared repo is cloned and identified in the UI; no repo code on host; sandbox created and cleaned up
Day 2 Security engine + agents	Demonstrate one vulnerability and a retestable patch	Pick one vulnerability class; deterministic scanner or targeted test; Red validation workflow; Blue root-cause and patch workflow; patch only after approval is wired; before/after tests with logs	Known vulnerability reproduced; Blue proposes a relevant patch; same test re-run reports the real result
Day 3 Manager + participation	Turn the pipeline into the VULNIGO experience	Manager via backend state transitions; structured outputs and call budgets; Spectator / Red / Blue modes; human messages; Need Backup; approval UI; score summary; events to pixel-art UI	User can spectate or join; both rounds run; backup works; patching waits for approval; report from stored evidence
Day 4 Integration + demo	Make the battle repeatable and reliable	Full end-to-end runs; invalid URL, clone failure, timeout tests; verify sandbox isolation and cleanup; UI state clarity; verify scores vs evidence; feature freeze; demo script and rehearsals incl. one failure/recovery	Demo repeatable; outputs evidence-backed; failures handled honestly; no known critical sandbox or approval bypass

Demo script

- Open the command centre and introduce VULNIGO; submit the prepared vulnerable repo and show sandbox status.
- Choose Join Blue (or Spectator for a shorter run); watch Red present a vulnerability with evidence.
- Review the finding: a candidate is not treated as confirmed until the test supports it.
- Blue explains root cause and proposes a patch; click Need Backup to show contextual help.
- Review the diff and explicitly approve; run the independent retest and show the real result.
- Show the final report, evidence links, unresolved risks and team scores.

> **Keep the demo honest.** Never stage a fake discovery. If a live model call fails, use a clearly labelled pre-recorded or deterministic run and say so.

04 Team and Responsibilities

Six members, six workstreams, two co-leads (Ishaan and Mariam). Roles are a proposed split based on the blueprint's layers; swap owners freely.

Member	Role / workstream	Owns
Ishaan	Co-lead + agent orchestration	Red, Blue and Manager agent service; prompts, structured outputs, model adapters and call budgets; Need Backup flow; integration across workstreams; shares team leadership with Mariam
Mariam	Co-lead + frontend / command centre	Connect the existing pixel-art UI to the backend; repo input, mode selection, agent activity, chat, Need Backup button, approval dialog, results and scoreboard; clear running / waiting / failed / completed states; shares team leadership with Ishaan
Askeri	Backend / battle controller	API endpoints, battle state machine, task routing, approvals, timeouts, retries, rate limits, event stream to the UI
APN	Sandbox + repo intake	GitHub URL validation (SSRF/path checks), clone and commit hash, disposable sandbox with non-root, no network, resource limits, command allowlist, cleanup
Ahmad	Security engine	Pick the one vulnerability class; build the intentionally vulnerable demo repo; deterministic scanner or safe proof test; before/after retest and regression checks
Sharjeel	Evidence, QA and demo	Evidence store and final report, score arithmetic from recorded facts, safety and failure-path testing, fallback run, demo script and rehearsals, this documentation

Day-by-day assignments

	Day 1	Day 2	Day 3	Day 4
Ishaan	Agent service skeleton, model adapters, schemas	Red validation and Blue patch workflows	Manager reports, human messages, Need Backup	Integration, bug triage, feature freeze
Mariam	Hook UI to backend status	Show findings and patch diff	Modes, chat, approval dialog, events to UI	UI state clarity, polish, demo flow
Askeri	Backend skeleton, health endpoint, state storage	State machine and test-run endpoints	Approvals, messages, backup endpoints, event stream	Timeout and API-failure handling
APN	URL validation, clone, sandbox create / cleanup	Run tests safely in sandbox, apply patch to working copy	Allowlisted tool interface for agent requests	Isolation, network and limits verification
Ahmad	Choose class, build vulnerable demo repo	Scanner / proof test; reproduce the issue; retest	Regression checks, confirmed vs hypothesis logic	Clean demo repo, repeatability
Sharjeel	Event and evidence schema	Log and preserve test evidence	Report and scoreboard from stored evidence	Evidence-vs-score checks, demo script, rehearsals

Ownership rule. Each exit criterion has one named owner. If a day's exit condition is at risk, the co-leads (Ishaan and Mariam) cut optional scope instead of carrying unfinished work forward, and Sharjeel and Ishaan re-verify that every reported claim still traces to recorded evidence.

05 Workflow Flowchart

End-to-end battle flow. Diamonds are decision gates; grey boxes are early exits that still end in a report.



Legend: navy = start / report, red = attack and retest, blue = defense and setup, gold = decision or inconclusive, grey = exit branch, green = verified outcome.

06 Tech Stack

Proposed choices per layer. The blueprint lets the team adapt tools to its existing codebase; items marked (suggested) are recommendations, not requirements.

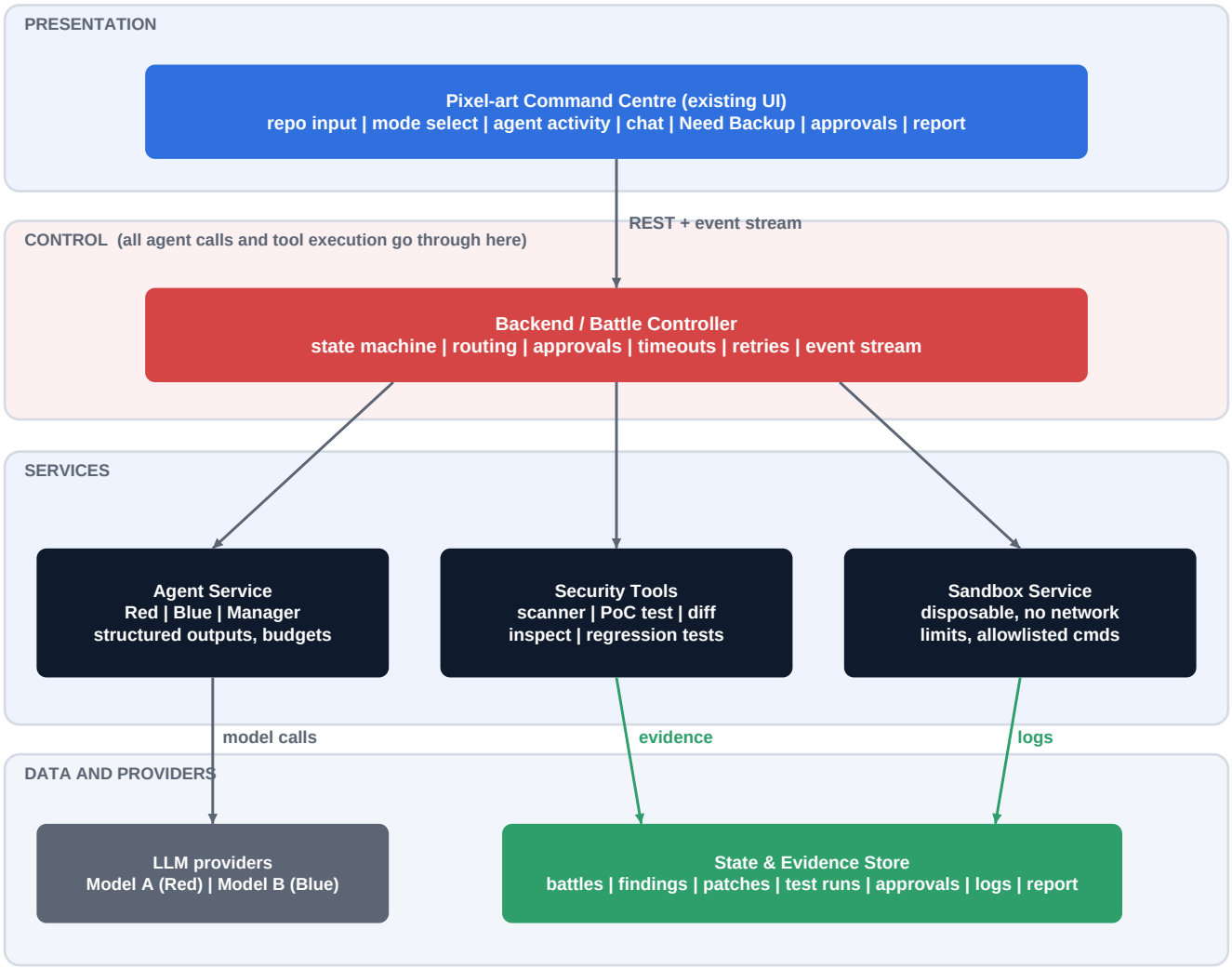
Layer	Proposed technology	Why / notes
Frontend	Existing pixel-art command centre (reuse as is); Server-Sent Events or WebSocket for the live event stream	Frontend never calls models directly; all calls go via the backend
Backend / Battle controller	Python + FastAPI (suggested); REST endpoints plus event stream; battle state machine in plain code	Predictable transitions, approvals, timeouts, retries and score math in code, not in the Manager model
Agent orchestration	LangGraph or LangChain (suggested) with provider adapters; Pydantic-validated structured outputs	Graph nodes map naturally to Red / Blue / Manager and the state machine; schema validation treats model output as untrusted
Models	Model A for Red, a different Model B for Blue; Manager reuses one or a cheaper model	Different families may reduce correlated blind spots but do not guarantee correctness
Security tools	Deterministic scanner or targeted proof test for one vulnerability class (e.g. Semgrep or custom tests, suggested); pytest / project test runner for regression	Pass/fail decisions come from deterministic tools, not agent opinion
Sandbox	Docker container, non-root, read-only source snapshot plus separate writable copy, network disabled, CPU/memory/PID/time limits (stronger isolation such as gVisor optional)	Core safety boundary; destroyed after each battle
Repo access	Git clone of validated GitHub URLs only; commit hash recorded	Reject local-file and private-network URLs (SSRF/path abuse)
State & evidence store	SQLite for the MVP (Postgres later) plus file storage for logs and diffs	Battles, findings, patches, test runs, approvals, final report
Config & secrets	Environment variables on the backend only; never passed into the sandbox	No API keys, cloud credentials or host env exposed to repo code
Observability	Structured logs of commands, approvals, patches and test outcomes	Supports audit and honest reporting

Structured agent outputs. Agents return fields such as finding_summary, file_path, line_range, confidence, test_plan, observed_result, patch_diff and limitations. The backend validates the schema and never executes a command just because an agent suggested it; every request is mapped to an allowlisted tool interface.

07 System Blueprint

Architecture, agent boundaries, state machine, API, data model, safety and testing

Architecture overview



Layer	Responsibilities
Pixel-art frontend	Repo input, participation mode, agent activity, chat/suggestions, Need Backup, approval dialog, results and scoreboard.
Backend / battle controller	API endpoints, state machine, task routing, approvals, session context, retries, rate limits, event stream.
Agent service	Red, Blue and Manager roles; model adapters; structured outputs; limited context windows; call budgets.
Security tools	Targeted checks, safe proof tests, patch diff inspection, regression tests.
Sandbox service	Disposable isolated environment, working copy, bounded commands, resource and network restrictions, cleanup.
State & evidence store	Repo metadata, battle state, findings, source refs, approvals, diffs, test logs, final report.

Agent responsibilities and boundaries

Agent	Does	Must not
Red (attacker)	Inspects allowed scope; names file, location, data flow and preconditions; prepares a safe bounded sandbox-only test; reports exact command, observed result and evidence; reruns the same test after the fix.	Treat a hypothesis as confirmed; test anything outside the local sandbox.
Blue (defender)	Receives only needed findings and evidence; explains root cause; proposes the smallest relevant patch with risks and assumptions; answers human suggestions and Need Backup.	Mark a fix successful just because a patch was generated or applied.
Manager (coordinator)	Tracks battle state; routes context; keeps an event timeline; links every claim to a test result or source location; compiles report and scoreboard.	Act as the sole security verifier; be called for every status change.

State machine

State	Allowed next state(s)
CREATED	REPOSITORY_VALIDATING
REPOSITORY_VALIDATING	SANDBOX_READY / FAILED
SANDBOX_READY	ATTACKING
ATTACKING	FINDINGS_REVIEW / ATTACK_INCONCLUSIVE
FINDINGS_REVIEW	DEFENDING / NO_CONFIRMED_FINDING
DEFENDING	AWAITING_APPROVAL / DEFENSE_INCONCLUSIVE
AWAITING_APPROVAL	PATCHING / PATCH_REJECTED
PATCHING	RETESTING / PATCH_FAILED
RETESTING	COMPLETED / UNRESOLVED / INCONCLUSIVE

API and event concepts (illustrative)

Endpoint	Purpose
POST /battles	Create a battle from a validated repo reference and selected scope
POST /battles/{id}/mode	Set spectator, Red participant or Blue participant
POST /battles/{id}/messages	Submit a human suggestion or question to the current team
POST /battles/{id}/backup	Request Blue Agent help using the active finding context
POST /battles/{id}/approvals	Approve or reject a proposed patch
GET /battles/{id}	Current state, events, findings and result summary
GET /battles/{id}/events	Stream agent activity and state changes to the command centre

Data recorded

Record	Important fields
Battle	ID, repository URL, commit hash, selected scope, mode, status, timestamps
Agent event	Agent role, event type, short message, timestamp, related finding ID, success/failure
Finding	ID, category, file and line, hypothesis, confirmation status, test reference, evidence
Patch	ID, changed files, diff, explanation, authoring agent, approval status, applied commit/hash
Test run	Command/tool, environment ID, exit code, output summary, start/end time, pass/fail/inconclusive
Approval	User decision, patch reference, timestamp, decision note

Security architecture

- Run all analysis and execution in an isolated, disposable environment; never on the application host.
- Non-root user, restricted mounts, no host sockets or secrets, strict CPU, memory, process and time limits.
- Outbound network disabled by default; allow only explicitly required local sandbox endpoints.
- Read-only source snapshot for analysis; separate writable working copy for patching.
- Keep logs of commands, approvals, patches and test outcomes; destroy the sandbox after the battle.
- Validate repo URLs; reject local-file, private-network and unintended schemes.
- Test only repositories the user owns or is authorized to assess; keep validation local, non-destructive and narrowly scoped.

Safe operating boundary. A successful test of one vulnerability does not certify the repository as secure. No scanning or exploitation of arbitrary public targets.

Evidence and status vocabulary

Status	Meaning
Confirmed	A bounded test reproduced the stated behavior and the evidence is recorded.
Potential / unconfirmed	Code suggests a weakness, but the test did not establish exploitability.
Fixed and retested	The original test no longer reproduces the issue and relevant regression checks pass.
Unresolved	The issue remains, the patch failed, or retesting did not establish a fix.
Inconclusive	The test could not run or produced insufficient evidence. This is not a pass.

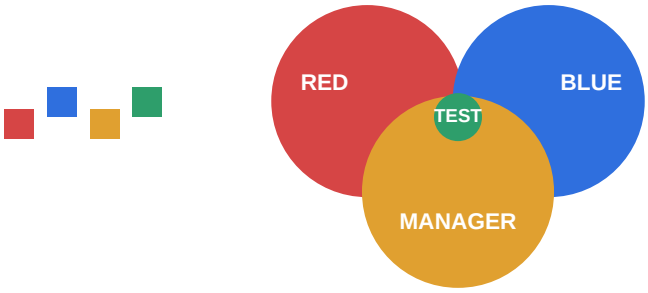
Testing and acceptance checklist

- Valid demo repo accepted and commit hash recorded; invalid input rejected with a useful message.
- Red produces a finding with source location and bounded test; confirmed vs hypothesis is distinguished.
- Blue explains root cause and produces a reviewable diff; approval is required before applying.
- Retest checks the original issue and expected behavior; inconclusive is never reported as fixed.
- Spectator, Join Red, Join Blue and Need Backup work for the prepared demo.
- Repo code cannot reach host files, secrets or runtime sockets; model-generated commands are validated before execution.
- Model/API failure creates no fake findings, approvals or fixes; sandbox data is cleaned up.

Risks and mitigations

Risk	Mitigation
Scope grows beyond four days	Freeze one vulnerability class, one demo repo and one complete battle; postpone progression features.
API credit or latency limits	Bounded calls, concise context, one model per role, deterministic tools for predictable checks.
Models share blind spots	Different model families if practical; static/targeted tests and independent retesting; do not claim diversity guarantees correctness.
False-positive findings	Require reproducible validation; label unconfirmed findings clearly.
Patch breaks other behavior	Review diff, require approval, run regression checks, keep changes inside the sandbox.
Sandbox escape or data exposure	Properly isolated disposable environment, strict limits, no secrets, security review.
Live demo failure	Known demo repo, repeated rehearsal, clearly labelled fallback recording or test run.

Product statement. VULNIGO - Gamified Repository Security. Learn repository security by taking part in AI-powered Red vs Blue battles. Investigate vulnerabilities, defend code, request AI backup and verify whether fixes actually work.



Attack and defense, coordinated by the Manager, verified by independent tests.